can also view memory as consisting of 32-bit (i.e. 4 byte) words

- the memory address of a word is the address of its first byte; e.g. the word with memory address 20 contains the bytes with addresses 20, 21, 22, and 23
- many architectures have an alignment restriction, requiring words to start at an address that is a multiple of the word size (in this case, a multiple of 4)
- RISC-V does not have such a restriction in general (although, in RV32I, machine language instructions must be aligned on a four-byte boundary)

little-endian versus big-endian

- when storing a 32-bit binary integer value in a memory word, do the bytes within the word with higher addresses store the leftmost ("higher-order") bits of the integer (this is the little-endian convention, and is used in our environment) or is it the other way around?

e.g. 97 as a 32-bit binary number is:
00000000000000000000000001100001
if store this in the memory word with address 20 (which consists of the bytes with addresses 20, 21, 22, and 23), which bits does the byte with address 23 get?

- becomes an issue when transmitting integers and other multi-byte items between systems that may use different conventions

RISC-V instructions

each instruction specifies a single, simple operation

the basic arithmetic instructions have three operands, a "destination" operand where the result goes, and 2 "source" operands

with the possible exception of the second source operand (the 3rd operand) which can sometimes be a numeric constant, operands must be registers

two example instructions:

```
sub s0, s1, s2
addi s0, s1, 42
```

example program

```
# function numbers for environment calls
                .equ SYS_exit, 93       # .equ, .section, .word, … are
                                        # “assembler directives”

# read-only data section
                .section .rodata
B:              .word 6                 # chose arbitrary values here (6, 10, 42)
C:              .word 10
D:              .word 42

# section for uninitialized data
                .section .bss
A:              .space 4                # 4 bytes = 1 word

# code section
                .section .text          # now comes the code …
                                        # our program is not very ambitious - it just
                                        # implements what we would write in the C
                                        # programming language as A = B + C + D;
                .global _start         # name “_start” must be externally visible;
                                        # used by the linker when creating the
                                        # executable file (this has a field storing
                                        # address at which execution should begin)
_start:         la t0, B                # puts the memory address that the name B
                                        # represents into the register t0
                                        # why t0?  no particular reason,  could have
                                        # used any of the “general purpose”
                                        # registers t0-t6 or s0-s11
```

```
lw s1, 0(t0)        # copies the contents of the memory word
                    # with address given by 0 + (contents of t0)
                    # into the register s1
                    # note that instead of this two
                    # instruction sequence, could have more
                    # simply used just “lw s1, B” … more to say
                    # about this later …
la t0, C            # and so on …
lw s2, 0(t0)
la t0, D
lw s3, 0(t0)

add s0, s1, s2      # can finally do the additions
add s0, s0, s3

la t0, A
sw s0, 0(t0)        # store result (which will be the integer 58)
                    # into the memory word that the name A
                    # represents (oddly, unlike what was noted
                    # for “lw” above, can’t just use “sw s0, A”)

li a0, 0            # put exit code (0) into register a0
li a7, SYS_exit     # when make a system call (using “ecall”)
                    # number in register a7 tells the OS
                    # (or in our case, the QEMU emulator)
                    # what service we want performed);
                    # register a0 can be used to provide an
                    # argument
ecall               # RISC-V system call (or “environment call”)
                    # instruction
```